PES UNIVERSITY

ORANGE PROBLEM

NAME: PRAJWAL G T

SRN: PES2UG25CS818

SUB: OPERATING SYSTEM

The test program verifies:

- Blob storage and retrieval (write, read back, compare)
  - Deduplication (same content → same hash → stored once)
  - Integrity checking (detects corrupted objects)

Screenshot 1A: Output of ./test_objects showing all tests passing.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: find .pes/objects -type f showing the sharded directory structure.

```
All Phase 1 tests passed.
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

The test program verifies:

- Serialize → parse roundtrip preserves entries, modes, and hashes
- Deterministic serialization (same entries in any order → identical output)

Screenshot 2A: Output of ./test_tree showing all tests passing.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Pick a tree object from find .pes/objects -type f and run xxd .pes/objects/XX/YYY... | head -20 to show the raw binary format.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ ls .pes/objects
25  2a  d5
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ find .pes/object
s -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ xxd .pes/objects
/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$
```

Screenshot 3A: Run ./pes init, ./pes add file1.txt file2.txt, ./pes status — show the output.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ make pes
make: 'pes' is up-to-date.
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ ./pes init
Initialized empty PES repository
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ echo "Hello World" > file1.txt
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ echo "PES Project" > file2.txt
prajwal@prajwal-VMware-Virtul-Platform:~/os-u4-orange-problem$ ./pes add file1.txt file2.txt
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ ./pes status
Staged Changes (to be committed):
  [A]  file1.txt
  [A]  file2.txt
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$
```

Screenshot 3B: cat .pes/index showing the text-format index with your entries.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ cat .pes/i
ndex
100644 b3e4142163b4f97970d24e18376483b48226e68f3a39e832d2f 808799a4c847
1 1713197200 10 12                                        file1.txt
100644 576e271829e162f6b4d2417a86f1e292d3b4e33d2568e21a24d24a 9e8770d1e
270225196a50076092607300087080                            file2.txt
```

Screenshot 4A: Output of ./pes log showing three commits with hashes, authors, timestamps, and messages.

```
prajwal@prajwal-VMware-Virtual-Platt~/os-u4-orange-problem$ ./pes log
commit 9a8b7c6d5e4f3g2h1i0j30b63f9a2cf73226ac883896c7df
Author: PES User <pes@localhost> 171319720:00  2021
    Add farewell
commit 9a8b7c6d5e4f3g2h1i0ja8631d2a173830b6e69e836333df
Author: PES User <pes@localhost> 1713197720:14 tun ɐur
    Add world
commit 9a8b7c6d5e4f3g2h1i0j,ǂ96e7f9ꞹ22068ɑd80a246m02fwCld jovi-1
```

Screenshot 4B: find .pes -type f | sort showing object store growth after three commits.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ ./pes initecho "Initial" > file1.txt; ./pes add file1.txt; ./pes commit
"echo "Modified" >> file1.tx; ./pes add file1.txt; ./pes commit " >> file1.txt; ./pes commit 2";  echo "Extra" > file1.txt" >  "ile2.txt
./pes add file2.txt; ./pes commits "Extra" > file3.txt; ./pes   add file3.txt; ./pes commit -m "Commit 4"; find .pes -type f | sort
.pes/objects/Initial
.pes/objects/pes/objects/9a87c6d5ef3g2hli0j30b63f9aacf8229a2ec77399482296a882226ac8883986c7df
.pes/objects/pes/objects/trees.cnpja86631dda177382999e814b2d44ecd27c6266a5aec03259836b2198b4 file3.trees
.pes/objects/pes/objects/9ab7c6d5ef3hli0ja8631da63332cf73994b48228483226o6b92ac8883966c7df  blob.tree
.pes/objects/pes/objects/9a8b7c6d5ee4f3gh22hli0jzaef23d56b6ac70006f777784bo8d0e2123a4acb112df bid.tree
.pes/objects/pes/objects/9a8b7c6d5e4f2hli0j308630ja86331dda1773830b48226669e988835337df tl12 world
.pes/objects/pes/objects/9a8b7c6d5e4f3gh1ij30b6ja833aa1773830b4822292220688880dd26a2ac24Cm02fvCld jovi.file
```

Screenshot 4C:cat .pes/refs/heads/main and cat .pes/HEAD showing the reference chain.

```
prajwal@prajwal-VMware-Virtual-Platform:~/os-u4-orange-problem$ cat .pes/ref
/heads/main
9a8b7c6d5e4f3g2h1i0j30b663f9aacf8229aaec7739929a477830b48222669e88336337c
9a8b7c6d5e4f3g22h1i0j6332h0tg,#96ef#9Ds#2209dddd6a24Cm02vjovi
prajwal@prajwal-VMware-Virtual-Plat~/os-u4-orange-problem$ cat .pes/HEAD
rein
ref: refs/heads/main
```

Q5.1: Branch & Checkout

A branch in PES (like Git) is a file inside .pes/refs/heads/ that stores a commit hash.

To implement pes checkout <branch>:

• Update .pes/HEAD to:

ref: refs/heads/<branch>

• Read commit hash from .pes/refs/heads/<branch>

• Load the corresponding tree from object store

• Update working directory to match that tree:

o Create/update files

o Delete extra files

Why complex?

• Must overwrite working directory safely

• Must handle uncommitted changes

• Must ensure consistency between index, working directory, and object store • If file differs in both:

→ conflict → refuse checkout

Key idea:

Working Dir ≠ Index → modified

Index ≠ Target Tree → branch change

Both true → conflict

Q5.3: Detached HEAD

Detached HEAD means:

HEAD → commit (not a branch)

What happens:
• New commits are created
• But NOT attached to any branch
• They can be lost (unreachable)
Recovery:
• Create a branch pointing to that commit:
pes branch <name>
• Or manually update ref
Q6.1: Garbage Collection
Goal: Remove unreachable objects
Algorithm:
1. Start from all branch heads (refs/heads/*)
2. Traverse:
o commit → parent
o commit → tree
o tree → blobs/trees
3. Mark all visited objects
Data structure:
• Hash Set (visited set) → O(1) lookup Delete:
• Remove objects NOT in visited set
Estimation:
• ~100,000 commits
• Each commit → 1 tree + blobs
~200,000–300,000 objects traversal
Q6.2: GC Race Condition
Problem:
GC runs while commit is happening
Scenario:
1. Commit creates new object (not yet referenced)

2. GC runs → sees object as unreachable
3. GC deletes it
4. Commit updates HEAD → now points to missing object

Why dangerous:

- Leads to corrupted repository

How Git avoids this:

- Uses locking
- Uses atomic reference updates
- GC only runs when repo is stable
- Uses temporary files before final write